

FeatureLeakageLens

Product Requirements Document — v0.2.0

Pre-training feature leakage auditor for tabular ML datasets

Field	Value
Status	Released — v0.2.0
Author	Sidharth Kriplani
Language	Python 3.10 / 3.11 / 3.12
License	MIT
Dependencies	pandas, numpy
Tests	13 unit tests, 4 test classes
PyPI	featureleakagelens

1. Problem Statement

Feature leakage is one of the most common and consequential failure modes in ML model development. Models that appear highly accurate during offline evaluation often fail in production because their training data includes information that would not have been available at prediction time.

Common leakage patterns:

- **Post-outcome columns** — a `payment_received_flag` set after the loan outcome is decided
- **Target proxies** — a `default_probability_v1` column computed from the same outcome variable
- **Future timestamps** — a feature generated or updated after the label event date
- **ID leakage** — high-cardinality identifiers that memorize training set labels
- **Train/test contamination** — systematic distribution differences suggesting a leaky split

These problems are well-known but rarely caught systematically. They are typically discovered late — during production monitoring, in a code review, or after a failed deployment. The cost of late discovery is high: wasted model development cycles, misleading business decisions, and eroded trust in ML outputs.

2. Goals and Non-Goals

Goals:

- Provide a single function call that audits a pandas DataFrame for six known leakage patterns
- Return structured, machine-parseable findings (PASS/WARN/FAIL/INSUFFICIENT_INPUT) before any model training
- Make the leakage review step systematic, documentable, and repeatable
- Surface definitive violations (temporal) vs. statistical suspicions (correlation, naming) with appropriate severity
- Produce three output formats: JSON for CI gates, Markdown for documentation, HTML for human review

Non-Goals:

- Proving causally that a feature caused model inflation (correlation $\neq$ leakage)
- Running or configuring experiments or data splits
- Replacing feature store governance, data contracts, or production monitoring
- Detecting nonlinear proxy leakage, multi-feature interaction leakage, or indirect leakage paths

- Making automatic go/no-go ship decisions — human review of findings is required

3. Target Users

User	Use case
ML Engineer / Data Scientist	Run audit before training; catch leakage before offline metrics mislead
Analytics Lead / Model Reviewer	Review audit report as part of model sign-off process
ML Platform / MLOps team	Integrate audit as CI gate in training pipeline; fail builds on FAIL status
Interview candidate	Demonstrate systematic ML engineering practice; show structured audit artifact

4. Check Specifications

Check	Config fields	Default threshold	Pass condition	Fail/Warn condition
Post-outcome name heuristic	target_col	—	No suspicious column names	WARN: any col name matches keyword
Target correlation scan	target_col, high_corr_threshold	0.95	$ r < \text{threshold}$ for all numerical features	WARN: $ r = \text{threshold}$ for any feature
Categorical proxy scan	target_col, cat_proxy_threshold	0.50	$\text{max_rate} - \text{min_rate} < \text{threshold}$	WARN: $\text{rate gap} \geq \text{threshold}$ for any cat
Temporal availability	outcome_time_col, feature_time_cols	—	All feature_ts $\leq$ outcome_ts	FAIL: any row has feature_ts $>$ outcome_ts
ID / proxy scan	id_threshold	0.95	$\text{n_unique}/\text{n_rows} < \text{threshold}$	WARN: any col has $\text{n_unique}/\text{n_rows} \geq \text{threshold}$
Split distribution scan	split_col, distribution_shift_threshold	0.10	All shift $<$ threshold	WARN: any col shift $\geq$ threshold; INSU

5. Input / Output Model

Input:

- **df**: pandas DataFrame — the training dataset to audit
- **config**: LeakageAuditConfig dataclass with the following fields:

Field	Type	Required	Description
target_col	str	Yes	Name of the binary target column
split_col	str None	No	Column with train/test labels (enables split distribution scan)
outcome_time_col	str None	No	Column with outcome event timestamp (enables temporal check)
feature_time_cols	dict[str, str]	No	Maps feature col -> timestamp col (enables temporal check)
high_corr_threshold	float	No	Pearson r threshold for target correlation scan (default 0.95)
cat_proxy_threshold	float	No	Target rate gap threshold for categorical proxy scan (default 0.50)
id_threshold	float	No	n_unique/n_rows threshold for ID/proxy scan (default 0.95)
distribution_shift_threshold	float	No	Shift threshold for split distribution scan (default 0.10)

Output — LeakageReport:

Field	Type	Description
status	str	PASS / WARN / FAIL / INSUFFICIENT_INPUT — worst finding across all checks
findings	list[LeakageFinding]	One entry per check per affected column
summary	dict	rows, columns, finding_count, warn_count, fail_count
to_json()	str	Full report as JSON string
to_markdown()	str	Human-readable Markdown report
to_html()	str	Self-contained HTML report with highlighted findings
save(path, stem)	None	Writes .json, .md, .html to path with given filename stem

6. Public API

Minimal usage:

```
from featureleakagelens import LeakageAuditConfig, audit_dataframe
report = audit_dataframe(df, LeakageAuditConfig(target_col='target'))
```

```
print(report.status) # PASS / WARN / FAIL
report.save('outputs/') # writes JSON, Markdown, HTML
```

Full config usage:

```
config = LeakageAuditConfig(
    target_col='defaulted',
    split_col='split',
    outcome_time_col='outcome_ts',
    feature_time_cols={'payment_received_flag': 'payment_received_ts'},
    high_corr_threshold=0.90,
    distribution_shift_threshold=0.15,
)
```

7. Test Plan

Test class	Tests	What is covered
FeatureLeakageLensTests	6	Core checks: missing target FAIL, name heuristic WARN, temporal FAIL, ID proxy WARN,
TargetCorrelationTests	2	High correlation warns; low correlation produces no finding
SplitDistributionTests	2	Large distribution shift warns; missing split_col gives INSUFFICIENT_INPUT
EvidenceTests	3	Temporal finding has future_rows evidence; report summary has expected keys; save() w

All 13 tests pass on Python 3.10, 3.11, 3.12. CI matrix runs on every push via GitHub Actions.

8. Success Criteria

Criterion	Target	Status
All 6 checks implemented	6/6	Done
Unit tests passing	13/13	Done
CI matrix (3.10/3.11/3.12)	All green	Done
Three output formats	JSON + MD + HTML	Done
Zero test dependencies beyond pandas + numpy	Yes	Done
Configurable thresholds via LeakageAuditConfig	All 4 thresholds exposed	Done
Explicit truth boundary in every report	Yes	Done
PyPI publish workflow (OIDC trusted publisher)	Configured	Done

Resume-safe claim: Built **FeatureLeakageLens**, a pre-training feature leakage auditor for tabular ML datasets that checks for post-outcome name heuristics, target correlation, categorical target-rate proxies, future timestamp leakage, ID/proxy columns, and train/test distribution shift, producing structured JSON/Markdown/HTML audit reports with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary.